FMX Academy

INFORMAÇÕES			
MÓDULO		**AULA**	**INSTRUTOR**
07	**ASP.NET Core Web API – Avançado - Padrões de Projeto, Arquitetura Limpa e Alta Performance**	**03 - Unit of Work – A Arte da Consistência e Alta Performance**	**Guilherme Paracatu**

1. A Garantia de Atomicidade

O que é o Padrão Unit of Work (Unidade de Trabalho)?

O **Unit of Work (UoW)** é um padrão de projeto de software que atua como um "coordenador de transações". A sua única responsabilidade é manter uma lista de tudo o que foi alterado (inserido, atualizado ou deletado) durante uma regra de negócio e, no final, enviar todas essas mudanças para o banco de dados em um **único lote (batch).**

A regra de ouro do UoW é a **Atomicidade**: ou *tudo* é salvo com sucesso, ou *nada* é salvo. Não existe "meio termo" no banco de dados.

Curiosidade Técnica: Se você usa o Entity Framework Core, você já usa o Unit of Work! O DbContext é, por baixo dos panos, uma implementação perfeita desse padrão (através do *Change Tracker* e do *SaveChanges*). O que faremos na nossa arquitetura é **encapsular** o DbContext para que a nossa camada de Aplicação (Handlers) possa decidir a hora exata de "apertar o botão de salvar", sem precisar conhecer o Entity Framework.

2. Quais dores do dia a dia o Unit of Work resolve?
Sem o Unit of Work, os sistemas de grande porte fatalmente sofrem com três problemas gravíssimos. É aqui que o padrão brilha:

💥 **Dor 1: O Desastre do "Salvamento Parcial" (Inconsistência de Dados)**
Imagine que o sistema precisa transferir a posse de 50 veículos de uma marca que foi comprada por outra. O seu código faz um foreach, atualiza o veículo e chama o Atualizar() do repositório (que tem um SaveChanges embutido).

- **O Problema:** O sistema salva os 20 primeiros veículos com sucesso. No 21º, a energia do servidor cai ou o banco de dados dá um *timeout*. O que acontece? O seu banco de dados fica corrompido! Metade da frota está na marca nova, metade na antiga. O cliente liga furioso.

- **A Solução do UoW:** O Repositório apenas "anota" a mudança na memória. Quando o foreach termina, o Handler chama _uow.Commit(). Se der erro no meio do caminho, nenhuma linha de código afetará o banco de dados físico.

💥 **Dor 2: O Gargalo de Performance (Excesso de I/O no Banco)**
Cada vez que o código chama SaveChanges(), a sua API abre uma conexão de rede com o SQL Server, inicia uma transação, executa o comando, comita, e fecha a conexão.

- **O Problema:** Fazer isso dentro de um loop de 1.000 registros gera 1.000 idas e vindas na rede. Isso derruba a performance da API e onera o servidor de banco de dados.

- **A Solução do UoW:** O EF Core acumula as 1.000 alterações na memória RAM da API. Quando o _uow.Commit() é chamado uma única vez no final, o EF Core gera um único *script* gigante e envia tudo em **uma única viagem** pela rede. A operação que demorava 10 segundos passa a demorar 200 milissegundos.

💥 **Dor 3: A Invasão de Responsabilidades (Falta de Coesão)**
Se deixarmos o SaveChanges dentro do método Adicionar() do Repositório, estamos dizendo que o Repositório toma a decisão de quando a regra de negócio acabou.

- **O Problema:** E se o seu Handler precisar adicionar um Veiculo no VeiculoRepository e, na mesma requisição, adicionar um log de auditoria no LogRepository? Se cada repositório salvar sozinho, teremos duas transações isoladas.

- **A Solução do UoW:** Os repositórios voltam a ser apenas coleções em memória (como o próprio nome diz). Eles não salvam nada. Quem diz *"a regra de negócio acabou, pode persistir"* é o **Handler** (o maestro), chamando o Unit of Work.

3. A Analogia: O Carrinho de Supermercado 🛒
"Programar sem Unit of Work é como ir ao supermercado, pegar um pacote de arroz, andar até o caixa, pagar e levar para o carro. Depois você volta, pega o feijão, vai no caixa, paga e leva pro carro. É exaustivo e lento! (Isso é o SaveChanges dentro do Repositório).

*O **Unit of Work** é o seu carrinho de compras. Os Repositórios vão colocando os itens dentro do carrinho (Change Tracker na memória). Você pode colocar, tirar, trocar a quantidade. Somente quando você termina toda a sua lista de compras (a Regra de Negócio no Handler), você vai ao caixa uma única vez e paga a conta inteira (_uow.Commit())."*

2. Implementando o Unit of Work

Passo 1: O Contrato (Na camada de Domain)
A regra de que o sistema precisa salvar as coisas de forma transacional pertence ao Domínio. Portanto, a interface fica isolada de tecnologias.

ITB.Domain/Interfaces/IUnitOfWork.cs

C#

```
using System.Threading.Tasks;

namespace ITB.Domain.Interfaces;

public interface IUnitOfWork
{
    // O método retorna true se as mudanças foram persistidas com sucesso no banco
    Task<bool> Commit();
}
```

Passo 2: A Implementação (Na camada de Infrastructure)

A infraestrutura é quem conhece o banco de dados. Aqui nós vamos "encapsular" o AppDbContext do Entity Framework para que ele atue como o nosso carrinho de compras.

ITB.Infrastructure/Persistence/UnitOfWork.cs

C#

```
using System.Threading.Tasks;
using ITB.Domain.Interfaces;

namespace ITB.Infrastructure.Persistence;

public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;

    public UnitOfWork(AppDbContext context)
    {
        _context = context;
    }

    public async Task<bool> Commit()
    {
        // SaveChangesAsync retorna o número de linhas afetadas no banco.
        // Se for maior que 0, significa que o Commit funcionou!
        return await _context.SaveChangesAsync() > 0;
    }
}
```

Passo 3: "A Grande Limpeza" dos Repositórios (Refatoração)

Chegou a hora de tirar a responsabilidade de salvar dos repositórios. Eles voltam a ser apenas coleções na memória!

Vá no **VeiculoRepository.cs** (e depois peça para os alunos fazerem o mesmo no MarcaRepository) e **apague** os SaveChangesAsync().

📄 **ITB.Infrastructure/Repositories/VeiculoRepository.cs**

C#

```
    public async Task Adicionar(Veiculo veiculo)
    {
        // Apenas "anota" na memória que este veículo deve ser inserido (Change
Tracker)
        await _context.Veiculos.AddAsync(veiculo);

        // ❌ REMOVIDO: await _context.SaveChangesAsync();
    }

    public Task Atualizar(Veiculo veiculo)
    {
        // ❌ REMOVIDO: _context.Veiculos.Update(veiculo);
        // ❌ REMOVIDO: await _context.SaveChangesAsync();

        // O método fica literalmente vazio ou pode ser removido da interface,
        // pois o EF Core já rastreia o objeto quando usamos o ObterPorId.
        // Mas vamos mantê-lo retornando Task.CompletedTask por semântica arquitetural.
        return Task.CompletedTask;
    }
```

Passo 4: O Maestro assume o controle (Na camada de Application)

Agora que os repositórios não salvam mais nada, o nosso **Handler** precisa chamar o Unit of Work no exato momento em que a regra de negócio acaba.

📄 **ITB.Application/Handlers/AtualizarVeiculoHandler.cs**

C#

```
using System.Threading.Tasks;
using ITB.Application.Commands;
using ITB.Domain.Core.Exceptions;
using ITB.Domain.Interfaces;

namespace ITB.Application.Handlers;

public class AtualizarVeiculoHandler : IHandler<AtualizarVeiculoCommand>
{
    private readonly IVeiculoRepository _veiculoRepository;
    private readonly IMarcaRepository _marcaRepository;
    private readonly IUnitOfWork _uow; // 👍 1. Injetamos o Unit of Work

    public AtualizarVeiculoHandler(
        IVeiculoRepository veiculoRepository,
        IMarcaRepository marcaRepository,
        IUnitOfWork uow)
    {
        _veiculoRepository = veiculoRepository;
```

```
        _marcaRepository = marcaRepository;
        _uow = uow;
    }

    public async Task Handle(AtualizarVeiculoCommand command)
    {
        // ... (Todo aquele código de busca e validações continua EXATAMENTE igual) ...
        var veiculo = await _veiculoRepository.ObterPorId(command.Id);
        // ... (validações de placa, etc) ...

        // Aplica as Intenções
        veiculo.AtualizarDados(command.Modelo, command.Ano);
        // ...

        // 🧠 2. O GRANDE FINAL: A transação atômica!
        if (!await _uow.Commit())
        {
            // Se o EF Core não conseguir salvar (ex: banco caiu, timeout), o sistema
avisa!
            throw new DomainException("Ocorreu um erro ao salvar as alterações no banco
de dados.");
        }
    }
}
```

Passo 5: Ensinando o contêiner
Para o sistema não explodir com aquele erro de *Unable to resolve service*, precisamos registrar a nossa nova interface no contêiner de Injeção de Dependência da API.

📄 **ITB.IoC/DependencyInjection.cs**

```
C#
// ... logo abaixo do registro dos Repositórios ...

// Registramos o Unit of Work com tempo de vida SCOPED (A mesma duração do DbContext e
da requisição HTTP)
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
```

3. Desafio Prático: Expandindo o Domínio ITB

Objetivo: Aplicar de forma autônoma todos os conceitos aprendidos até o momento (Domínios Ricos, Repositórios, Injeção de Fatos e Unit of Work).

Cenário: Atualmente, nosso sistema trata o "Modelo" do veículo como uma simples string. Para melhorar a qualidade dos dados, precisamos que o **Modelo** se torne uma entidade independente e vinculada a uma **Marca**.

Sua Missão: Implementar o fluxo completo para a nova entidade Modelo, seguindo os requisitos abaixo:

1. **Criação da Entidade:** O Modelo deve possuir Id, Nome, MarcaId e Ativo. Utilize o padrão de Domínio Rico (construtores protegidos e validações internas).

2. **Repositório Atômico:** Implementar o IModeloRepository e sua versão concreta. Lembre-se: o repositório **não deve** salvar alterações no banco (retire o SaveChangesAsync).

3. **Handler e Unit of Work:** Criar o AdicionarModeloHandler. Ele deve validar se a MarcaId informada realmente existe no banco antes de criar o modelo. O salvamento deve ser feito estritamente através do IUnitOfWork.Commit().

4. **Ajuste de Hierarquia:** Refatore a entidade Veiculo para que ela deixe de receber uma string Modelo e passe a receber um int ModeloId, apontando para a sua nova tabela.